Итоговое задание по модулю 3

Юхнов Тимофей Максимович

Нереляционные базы данных

13 марта 2026 г.

Содержание

1 Схема базы данных 2

2 Реализация шардинга и обоснование выбора 2

3 Нагрузочное тестирование и визуализация 3

4 Репозиторий с кодом 4

1 Схема базы данных

В рамках проекта используется структура базы данных `university` из итогового задания по модулю 2 прошлого семестра. А конкретно коллекция `grades` (оценки студентов). Документы в коллекции имеют плоскую структуру и описывают факт получения оценки студентом по определенному курсу.
Структура документа:

- `student_id` - уникальный идентификатор студента (например, `"S1234"`).
- `course_id` - идентификатор учебного курса (например, `"C5"`).
- `teacher_id` - идентификатор преподавателя (например, `"T2"`).
- `grade` - оценка по десятибалльной шкале (от 1 до 10).

2 Реализация шардинга и обоснование выбора

Для обеспечения горизонтального масштабирования был развернут кластер MongoDB с использованием Docker Compose.
Архитектура кластера:

- **Маршрутизатор (mongos):** 1 контейнер (точка входа)
- **Конфиг-сервер (configsvr):** 1 контейнер
- **Шарды (shard1, shard2):** 2 независимых контейнера для распределенного хранения данных

Выбор ключа и типа шардинга: В качестве ключа шардирования было выбрано поле `student_id` и использовано range-based шардирование.

Команда инициализации:

```
sh.shardCollection("university.grades", { student_id: 1 })
```

Обоснование:

1. **Оптимизация чтения:** Основные запросы в клиентском приложении (поиск оценок студента и расчет среднего балла) выполняются с фильтрацией по точному совпадению `student_id`. Благодаря диапазонам, `mongos` точно знает, на каком шарде лежат данные конкретного студента, и отправляет запрос напрямую туда, минуя остальные узлы. Это минимизирует сетевые задержки.
2. **Равномерное распределение записи:** Классическая проблема диапазонного шардирования это горячие узлы, куда запись просиходит чаще всего. Однако в данном проекте генерация `student_id` происходит случайным образом (от S0 до S9999). Как и в реальной жизни, оценки получает не слудующий по айди студент, а один любой из существующих. Благодаря этому новые документы изначально равномерно попадают в разные куски диапазона, обеспечивая балансировку нагрузки на запись между `shard1` и `shard2`.

3 Нагрузочное тестирование и визуализация

Для оценки производительности системы был разработан скрипт на Python с использованием библиотеки `pymongo` и пула потоков `ThreadPoolExecutor`.

Методология:

Тестировались две базовые операции: вставка (`insert_one`) и чтение (`find_one`). Измерения времени выполнения проводились для объемов в 1000 и 10000 документов при уровне одновременных потоков равном 1, 10, 50 и 100.

Анализ результатов:

- **Запись:** Переход от последовательной записи (1 поток) к многопоточной (10 потоков) ускоряет процесс в 4–5 раз. Оптимальная производительность (около 8–9 секунд на 10 000 записей) достигается при 50 потоках. Дальнейшее увеличение потоков до 100 не дает прироста скорости записи, так как достигается предел пропускной способности сети и дисковой подсистемы Docker контейнеров.

- **Чтение:** Операции чтения в однопоточном режиме выполняются медленнее записей (около 57 секунд на 10 000 запросов). Однако распараллеливание до 10–50 потоков резко снижает время до 10–12 секунд. При 100 потоках наблюдается небольшой рост времени до 14 секунд, что свидетельствует о начале конкуренции за ресурсы.

Вывод: Кластер корректно масштабирует нагрузку. Оптимальный размер пула потоков для данной конфигурации составляет от 10 до 50.

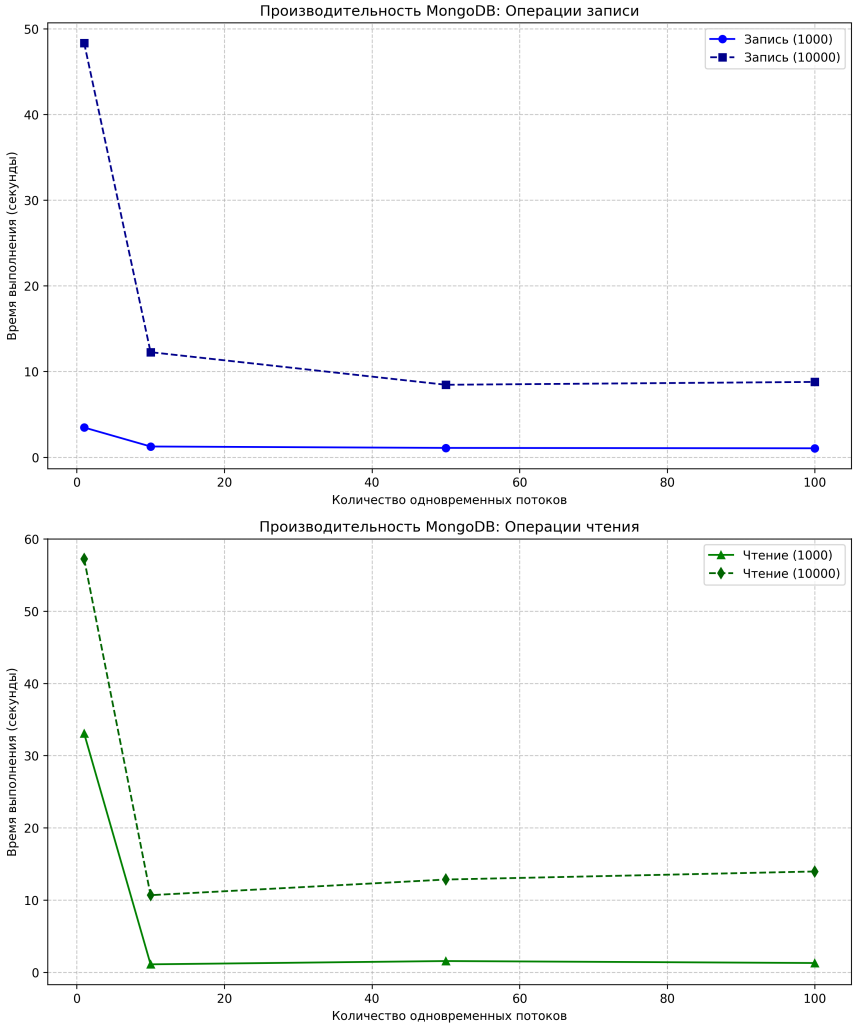

Рис. 1: Графики производительности: операции записи и чтения

4 Репозиторий с кодом

Полный исходный код проекта доступен в репозитории:

Ссылка на репозиторий: https://github.com/timyuk/nosql_3